

Die Vitalzeichen einer Plattform: Wie man mit den Four Golden Signals den Gesundheitszustand von frag.jetzt überwacht

Exposé zur Bachelorarbeit im Studiengang Informatik

Max Mustermann

25. Januar 2026

Zusammenfassung

Moderne Webplattformen wirken nach außen oft stabil und reaktionsschnell, doch ihr innerer Zustand bleibt im laufenden Betrieb häufig verborgen. Dieser Artikel zeigt, wie Observability dabei hilft, verteilte Systeme wie die Open-Source-Plattform *frag.jetzt* als Ganzes zu verstehen und ihren „Gesundheitszustand“ sichtbar zu machen. Anhand der Four Golden Signals wird diskutiert, welche technischen Anzeichen tatsächlich aussagekräftig sind, warum isoliertes Monitoring nicht ausreicht und wie Metriken, Logs, Traces und intelligentes Alerting zusammenspielen müssen, um handlungsrelevante Einblicke zu liefern. Im Fokus steht dabei weniger die Theorie einzelner Werkzeuge als vielmehr die Frage, wie aus Rohdaten Entscheidungen werden, die Betrieb, Entwicklung und Produktverantwortung gleichermaßen unterstützen.

beiten aus dem Umfeld cloud-nativer Anwendungen berichten, dass strukturierte Observability-Praktiken die Incident-Analyse unterstützen und Reaktionszeiten verkürzen können [5, 2]. Die zentrale Frage lautet also: Welche „Vitalzeichen“ zeigen zuverlässig, ob ein Dienst gesund ist – und welche Daten helfen tatsächlich, wenn es kritisch wird?

Diese Arbeit trägt den Titel: *Die Vitalzeichen einer Plattform: Wie man mit Observability den Gesundheitszustand von frag.jetzt überwacht*. Im Mittelpunkt steht die Open-Source-Plattform *frag.jetzt*, deren Kernwert im produktiven Betrieb davon abhängt, dass Nutzerfragen entlang eines verteilten Anfragepfads schnell, korrekt und nachvollziehbar beantwortet werden [6]. Trotz einer soliden Entwicklungs- und Deployment-Infrastruktur fehlt derzeit ein durchgängiges Observability-Konzept, das Metriken, Logs, Traces und Alarmierung konsistent entlang dieser End-to-End-Kette verbindet [6, 5]. Dadurch bleiben Zusammenhänge zwischen Nutzererlebnis, Backend-Verarbeitung und externen Abhängigkeiten nur eingeschränkt sichtbar.

1 Einleitung: Die Vitalzeichen einer Plattform

Eine Frage wird abgeschickt, die Oberfläche wirkt kurz wie eingefroren, und irgendwo im Hintergrund beginnt eine Kette aus Diensten zu arbeiten. Kommt die Antwort in Sekunden – oder bleibt der Nutzer vor einem Ladeindikator hängen? In cloud-nativen Systemen entscheidet sich die Qualität einer Anwendung nicht nur im Quellcode, sondern im Betrieb: in Lastspitzen, in Randfällen, an Schnittstellen zu externen Diensten. Genau dort steigt die Komplexität, weil verteilte Architekturen zwar flexibel skalieren und Komponenten unabhängig weiterentwickeln lassen, zugleich aber die Überwachung und Fehlersuche spürbar erschweren [1, 2]. Besonders Microservices erzeugen Fehlerbilder, deren Ursache nicht mehr eindeutig „einem“ Baustein zugeordnet werden kann, weil Abhängigkeiten und Nebenwirkungen über Service-Grenzen hinweg wirken [3].

Hier setzt Observability an: Nicht als neues Buzzword, sondern als Versuch, den inneren Zustand eines Systems aus seinen messbaren „Ausgaben“ zu verstehen. Dazu werden Metriken, Logs und Traces gemeinsam betrachtet, um Verhalten, Fehler und Leistungsmerkmale in Beziehung zu setzen [4, 1]. Empirische Ar-

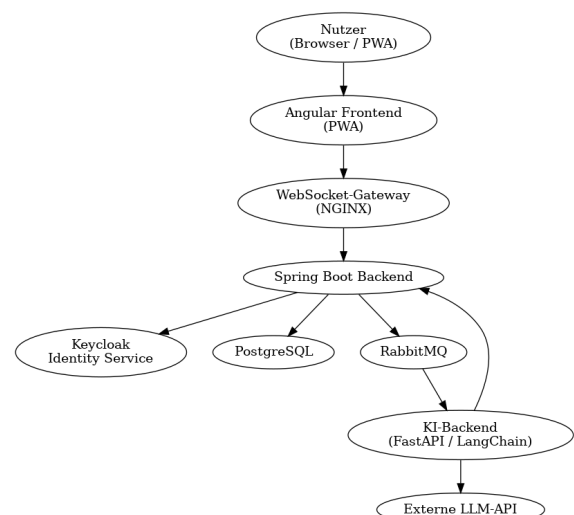


Abbildung 1: Vereinfachter Anfragepfad bei *frag.jetzt*: Von der Nutzerfrage bis zur Antwort über mehrere Komponenten hinweg. Quelle: Eigene Darstellung in Anlehnung an die Architekturübersicht im Projekt-Repository [6].

1.1 Warum gerade jetzt? Drei Beobachtungen aus dem Forschungsstand

Erstens: Die Literatur beschreibt wiederkehrend, dass klassische Monitoring-Ansätze in dynamischen, verteilten Umgebungen an Grenzen stoßen, weil sie vor allem bekannte Fehlerzustände mit statischen Schwellenwerten überwachen, aber bei unbekannten Störungen wenig Hilfestellung liefern [4, 2]. Zweitens: Moderne Observability wird in Studien als Kombination mehrerer Telemetriearten verstanden – erst ihr Zusammenspiel ermöglicht robuste Ursachenanalyse [7, 2]. Drittens: Alarmierung gilt als besondere Schwachstelle, weil zu viele Fehlalarme („Alarmmüdigkeit“) operative Entscheidungen verschlechtern; daher werden Priorisierung und datenbasierte Verfahren zur Reduktion irrelevanter Alarme als Forschungsschwerpunkte diskutiert [8].

Damit rückt eine pragmatische Leitfrage nach vorn: Welche Signale helfen im Alltag der Plattform wirklich – für DevOps, Entwicklung und Produktverantwortliche gleichermaßen?

2 Vier Golden Signals als Vitalzeichen: hilfreich, aber nicht selbsterklärend

Ein oft genutzter Startpunkt sind die *Four Golden Signals* aus Googles Site Reliability Engineering: *Latency, Traffic, Errors* und *Saturation* [9]. Sie wirken wie ein medizinischer Schnellcheck: Sie liefern Symptome, aber noch keine Diagnose. Genau diese Stärke macht sie so attraktiv, denn sie sind universell genug, um auf nahezu jeden nutzerorientierten Dienst angewandt zu werden [9]. Gleichzeitig wird betont, dass ihre Wirksamkeit davon abhängt, wie sie in Alarmierung, Entscheidungsprozesse und betriebliche Abläufe eingebettet werden [9].

- **Latenz:** Wie lange dauert es, bis eine Anfrage bearbeitet ist? Hohe oder schwankende Zeiten deuten auf Engpässe hin [9].
- **Traffic:** Wie viel Last trifft auf einen Dienst? Einbrüche können ebenso verdächtig sein wie Spitzen [9].
- **Fehler:** Wie viele Anfragen schlagen fehl – technisch oder funktional? [9]
- **Sättigung:** Wie ausgelastet sind limitierende Ressourcen wie CPU, Speicher, I/O oder Warteschlangen? [9]

Die entscheidende Zuspitzung lautet: Ein korrekt gemessener Wert ist noch keine handlungsrelevante Information. Ein Alarm, der niemandem sagt, *wer* betroffen ist, *wie stark* der Schaden ausfällt und *welche* nächsten Schritte plausibel sind, bleibt nur Lärm [8].

3 Der Patient: *frag.jetzt* als verteiltes System mit kritischem Pfad

frag.jetzt ist eine Cloud-native Q&A-Plattform, die interaktive Fragerunden unterstützt und zusätzlich KI-gestützte Assistenzfunktionen integriert [6]. Der zentrale Pfad ist aus Nutzersicht simpel: *Frage stellen* → *Antwort erhalten*. Technisch wird dieser Ablauf jedoch über mehrere Komponenten umgesetzt, darunter ein Angular-Frontend (SPA/PWA), ein WebSocket-Gateway (NGINX), ein Spring-Boot-Backend, Authentifizierung über Keycloak, Persistenz in PostgreSQL sowie ein KI-Backend (u. a. FastAPI/Langchain) mit externen LLM-Abhängigkeiten; asynchrone Entkopplung kann über RabbitMQ erfolgen [6]. Jeder zusätzliche Hop ist ein potenzieller Latenzpunkt, jeder externe Dienst eine Fehlerquelle, und jede Warteschlange kann sich unbemerkt füllen – bis es für Nutzer sichtbar wird.

3.1 Welche Fragen muss ein Betriebsteam in Sekunden beantworten können?

Statt Telemetrie als Selbstzweck zu sammeln, beginnt Observability mit Fragen. Für *frag.jetzt* drängen sich insbesondere diese Fragen auf:

- **Nutzererlebnis:** Wird die Antwort schnell genug sichtbar – oder entsteht „gefühlte“ Langsamkeit im Frontend?
- **Fehlerlokalisierung:** Liegt das Problem im Spring-Boot-Backend, in der Datenbank, im Auth-Flow oder in der externen KI-API?
- **Abhängigkeiten:** Verursacht eine schwankende externe KI-Latenz Kaskadeneffekte, etwa Stau in Queues oder erschöpfte Verbindungspools?
- **Priorisierung:** Betrifft die Störung wenige Anfragen oder das Kernfeature? Ist sofortiges Eingreifen nötig?

Gerade diese Verbindung aus Technik und Nutzungskontext wird in der Literatur als Hebel beschrieben, um Observability im Betrieb wirksam zu machen [7, 5].

4 Werkzeuge als Diagnosegeräte: Metriken, Logs, Traces – und der Streit um den Stack

In der Praxis existiert kein „ein“ Observability-Tool. Vielmehr werden spezialisierte Bausteine kombiniert: Metriken liefern Trends und Grenzwerte, Logs zeigen Ereignisdetails, Traces rekonstruieren den Weg einzelner Anfragen durch verteilte Systeme [7, 2]. Distributed Tracing wird dabei besonders für Microservices als essenziell beschrieben, weil es Latenzengpässe und Abhängigkeiten entlang des gesamten Anfragepfads sichtbar macht [10].

Für die Werkzeugauswahl stehen in Open-Source-Ökosystemen mehrere etablierte Optionen bereit:

- **Prometheus** zur Erfassung und Abfrage zeitbasierter Metriken (inkl. regelbasiertem Alerting) [11].
- **Grafana** zur Visualisierung und Korrelation verschiedener Datenquellen [5].
- **Elastic Stack (ELK)** als verbreitete Lösung für zentrale Log-Aggregation und Analyse [11].
- **Jaeger** für Distributed Tracing und End-to-End-Latenzanalyse [10].
- **OpenTelemetry** als herstellerneutraler Standard zur Instrumentierung für Metriken, Logs und Traces [10].

Vergleichsstudien diskutieren diese Toolklassen entlang von Kriterien wie Telemetrieabdeckung, Integrationsfähigkeit, Skalierbarkeit und Betriebsaufwand [5, 11]. Daraus ergibt sich eine praktische, aber forschungsnahe Auswahlfrage: Welche Kombination passt am besten zu den Anforderungen und Ressourcen von *frag.jetzt*?

Entscheidungsfrage	Warum sie zählt
Metriken-first oder Logs-first?	Metriken zeigen Trends, Logs liefern Details — beeinflusst Kosten und Diagnosezeit. [7]
Tracing-Pflicht?	Ohne End-to-End-Traces bleiben Engpässe und Abhängigkeiten in Microservices schwer greifbar. [10]
Instrumentierung mit Standard?	OpenTelemetry entkoppelt Erfassung vom Backend und reduziert Risiko eines Vendor Lock-ins. [10]

Tabelle 1: Leitfragen zur Stack-Auswahl für Observability. Quelle: Eigene Darstellung auf Basis der Literatur [7, 10, 5].

5 Forschungsfragen: Von Vitalzeichen zu Entscheidungen

Aus Problemstellung, Architektur und Forschungsstand leiten sich drei Forschungsfragen ab:

1. Wie können die Four Golden Signals für die spezifischen Services von *frag.jetzt* (PWA-Frontend, Spring Boot Backend, PostgreSQL, KI-Services) konkret definiert und instrumentiert werden? [9, 1]
2. Welche Observability-Stack-Kombination (Prometheus/Grafana, ELK Stack, Jaeger) ist am besten geeignet für die Monitoring-Anforderungen von *frag.jetzt*? [5, 11]
3. Wie kann ein intelligentes Alert-System entwickelt werden, das False Positives minimiert und handlungsorientierte Einblicke für verschiedene Rollen (DevOps, Entwicklung, Product Owner) bereitstellt? [8]

6 Methodik als Storyline: Sechs Schritte vom Symptom zur Therapie

Die Arbeit folgt einem artefaktorientierten Forschungsdesign mit konzeptionellem und prototypi-

sem Schwerpunkt. Statt jeden Baustein bis ins letzte Detail zu erklären, wird eine nachvollziehbare „Diagnosekette“ aufgebaut – von der Definition der Vitalzeichen bis zu Runbooks, die im Incident-Fall konkrete Schritte liefern.

6.1 Schritt 1: Golden Signals pro Komponente konkretisieren

Die Golden Signals werden für jedes Teilsystem so interpretiert, dass sie sowohl aus Technik- als auch aus Nutzungsperspektive verständlich sind [9]. Für ein PWA-Frontend kann „Latenz“ etwa als wahrgenommene Antwortzeit erscheinen, während im Backend zusätzliche Perspektiven (API-Latenz, Queue-Wartezeiten, Datenbank-Response) relevant werden [1].

6.2 Schritt 2: Tool-Stacks vergleichen und begründet auswählen

Da eine vollständige praktische Erprobung aller Werkzeuge im Rahmen der Arbeit unrealistisch ist, wird der Vergleich primär theoretisch-konzeptionell durchgeführt. Bewertet werden u. a. Telemetrieabdeckung, Integrationsfähigkeit, Betriebsaufwand und Reifegrad [5, 11]. Ergebnis ist eine begründete Stack-Empfehlung für *frag.jetzt*.

6.3 Schritt 3: Instrumentierung entlang des Anfragepfads

OpenTelemetry dient als einheitliche Instrumentierungsschicht, um Metriken, Logs und Traces konsistent zu erfassen und korrelierbar zu machen [10]. Entscheidend ist dabei nicht die Menge an Daten, sondern die Fähigkeit, eine Nutzeranfrage über Systemgrenzen hinweg wiederzufinden.

6.4 Schritt 4: Dashboards, die Rollen statt Datenquellen abbilden

Dashboards werden rollenbasiert konzipiert, damit nicht jede Zielgruppe dieselben Rohdaten interpretieren muss. Während DevOps eine schnelle Systemübersicht benötigt, interessiert die Entwicklung oft der Drilldown bis zum Trace, und Product Owner benötigen eine Sicht auf Nutzerwirkung und Priorität [5].

6.5 Schritt 5: Alarmierung, die Relevanz und Dringlichkeit kennt

Ein Alerting-Ansatz wird entworfen, der sich an den Golden Signals orientiert und zugleich Mechanismen zur Reduktion irrelevanter Alarmer berücksichtigt, etwa Aggregation mehrerer Signale, zeitliche Glättung und einfache Anomalieerkennung [8]. Ziel ist eine Alarmierung, die nicht nur „etwas ist anders“ ruft, sondern beantwortet: *Ist Handeln nötig, wen betrifft es, und was ist der nächste sinnvolle Schritt?*

6.6 Schritt 6: Runbooks als operationalisierte Erfahrung
Runbooks werden für wiederkehrende Incident-Szenarien erstellt und verknüpfen Alerts mit Diagnose- und Handlungsanweisungen. Damit wird Wissen aus der Telemetrie in operative Routine überführt, was in Observability-Ansätzen als wichtiger Baustein zur Verkürzung der Reaktionszeit gilt [5, 2].

7 Ergebnisartefakte: Was am Ende „in der Hand“ liegt

Aus den Schritten ergeben sich vier geplante Artefakte:

1. ein wissenschaftlicher Bericht zur Strategie, Toolauswahl und Alarmierungskonzeption,
2. eine lauffähige Observability-Plattform (z. B. Prometheus/Grafana) für *frag.jetzt*,
3. rollenbasierte Dashboards,
4. exemplarische Runbooks für typische Incident-Situationen.

8 Fazit und Ausblick: Wenn Vitalzeichen zu Vertrauen werden

Observability ist dann gelungen, wenn aus Telemetrie Vertrauen entsteht: Vertrauen, dass Nutzer nicht im Dunkeln warten, dass Fehler nicht nur entdeckt, sondern erklärt werden können, und dass Betriebsteams handlungsfähig bleiben, auch wenn externe Abhängigkeiten schwanken. Die Literatur zeigt, dass gerade in cloud-nativen Systemen strukturierte Observability-Praktiken die Analyse von Störungen unterstützen und operative Reaktionszeiten verkürzen können [5, 2]. Für *frag.jetzt* wird dieser Gedanke in eine konkrete Strategie übersetzt, die Golden Signals als Einstieg nutzt, aber durch Kontext, Tracing-Korrelation, rollenbasierte Sichten und priorisierte Alarmierung in Richtung betrieblicher Nutzbarkeit erweitert [9, 10, 8].

Im Ausblick bieten sich insbesondere SLO-basierte Steuerung und eine schrittweise Reifegradsteigerung an, um technische Symptome noch konsequenter mit Nutzerwirkung zu verbinden [9, 5]. Damit würden aus vier Vitalzeichen nicht nur Messwerte, sondern ein belastbarer Gesundheitscheck für eine Plattform, die von Interaktion lebt.

Abkürzungsverzeichnis

API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
CPU	Central Processing Unit
ELK	Elasticsearch, Logstash, Kibana
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
KI	Künstliche Intelligenz
NLP	Natural Language Processing
OTel	OpenTelemetry
PWA	Progressive Web App
Q&A	Question and Answer
REST	Representational State Transfer
SLO	Service Level Objective
SPA	Single-Page Application
SRE	Site Reliability Engineering
UI	User Interface

Quellen und weiterführende Literatur

- [1] U. Faseeha u. a. „Observability in Microservices: An In-Depth Exploration of Frameworks, Challenges, and Deployment Paradigms“. In: *IEEE Access* 13 (2025), S. 72011–72035. DOI: 10.1109/ACCESS.2025.3562125.
- [2] Kowshik Sakinala. „Monitoring and Observability for Cloud-Native Applications“. In: *Journal of Computer Science and Technology Studies* 7.8 (2025), S. 101–115. URL: <https://al-kindipublishers.org/index.php/jcsts/article/view/10459>.
- [3] S. Fischer u. a. „An Overview of Microservice-Based Systems Used for Evaluation in Testing and Monitoring: A Systematic Mapping Study“. In: (2024). DOI: 10.1145/3644032.3644445.
- [4] Cindy Sridharan. *Distributed Systems Observability*. O'Reilly Media, 2018. URL: <https://unlimited.humio.com/rs/756-LMY-106/images/Distributed-Systems-Observability-eBook.pdf>.
- [5] Luca Giamattei u. a. „Monitoring tools for DevOps and microservices: A systematic grey literature review“. In: *Journal of Systems and Software* 208 (2024), S. 111906. DOI: 10.1016/j.jss.2023.111906.
- [6] TH Mittelhessen und arsnova. *frag.jetzt – Open-Source Q&A-Plattform*. 2025. URL: <https://gitlab.arsnova.eu/arsnova/frag.jetzt/-/blob/staging/README.md> (besucht am 20.01.2026).

-
- [7] Olumese Anthony Abieba u. a. „Advanced Observability and Monitoring Practices for Enhancing Production Environment Reliability“. In: (2025). URL: <https://worldscientificnews.com/wp-content/uploads/2025/06/WSN-204-2025-270-296.pdf>.
 - [8] Fatemeh Jalalvand u. a. „Alert prioritisation in security operations centres: A systematic survey on criteria and methods“. In: *ACM Computing Surveys* 57.2 (2024), S. 1–36. DOI: 10.1145/3695462.
 - [9] Rob Ewaschuk. *Monitoring Distributed Systems*. <https://sre.google/sre-book/monitoring-distributed-systems/>. Kapitel 6 in *Site Reliability Engineering*, herausgegeben von Betssy Beyer, Google. 2017.
 - [10] Carlos Albuquerque und Filipe F Correia. „Tracing and Metrics Design Patterns for Monitoring Cloud-native Applications“. In: *arXiv preprint arXiv:2510.02991* (2025). URL: <https://arxiv.org/abs/2510.02991>.
 - [11] A. Banerjee und R. Singh. „A Comparative Analysis of System Monitoring Architecture: Evaluating Prometheus, ELK Stack, and Custom Dashboards for Performance and Scalability“. In: *International Journal of Computer Applications* (2024). DOI: 10.21275/SR25917095133.

Hinweis zur KI-Nutzung: Bei der Erstellung dieses Exposés wurden KI-Tools (ChatGPT 5.2, Google Gemini Pro 3, Consensus, Elicit) zur Unterstützung bei Recherche und Formulierung eingesetzt. Die inhaltliche Verantwortung liegt beim Autor.